

나만의 사이트 데이터베이스 모델링 실습지 모범답안

예시 사이트: 온라인 도서 대여·리뷰 사이트

1. 나의 사이트 정하기

구현하고 싶은 사이트

사이트 이름:

북루프(BookLoop)

사이트 종류:

도서 대여·리뷰 사이트

사이트의 주요 목적:

회원이 도서를 검색하고 여러 권의 책을 한 번에 대여하며,
대여한 도서에 별점과 리뷰를 작성할 수 있도록 한다.

주요 사용자:

도서를 대여하고 읽은 후 리뷰를 작성하려는 회원

사용자가 할 수 있는 주요 활동:

- 회원가입과 로그인
 - 도서 검색과 상세정보 확인
 - 여러 권의 도서 대여
 - 대여 현황과 반납 예정일 확인
 - 도서 별점과 리뷰 작성
-

도입 활동

활동 0. 사이트에서 관리할 정보 생각하기

나의 사이트에서 관리할 정보

1. 회원 이름
 2. 로그인 아이디
 3. 도서 제목
 4. 저자
 5. 도서 대여료
 6. 대여일
 7. 반납 예정일
 8. 반납 상태
 9. 리뷰 내용
 10. 별점
-

1단계. 사이트 요구사항 분석하기

활동 1. 사이트 운영 규칙 작성하기

내 사이트에는 여러 명의 회원이 가입할 수 있다.

회원은 여러 번 도서를 대여할 수 있다.

한 번의 대여에는 여러 권의 도서가 포함될 수 있다.

같은 도서는 여러 회원의 대여 기록에 포함될 수 있다.

사이트에서는 도서 제목, 저자, 현재 대여료와 대여 가능 여부를 관리한다.

대여한 각각의 도서에는 대여 당시 금액과 반납 상태를 저장한다.

회원은 자신이 읽은 도서에 별점과 리뷰를 작성할 수 있다.

한 도서에는 여러 회원이 리뷰를 작성할 수 있다.

활동 2. 요구사항에서 중요한 명사 찾기

번호	중요한 명사	사용되는 문장
1	회원	여러 명의 회원이 가입할 수 있다.
2	도서	여러 권의 도서를 대여할 수 있다.
3	대여	회원은 여러 번 대여할 수 있다.
4	대여도서	한 번의 대여에는 여러 도서가 포함된다.
5	리뷰	회원은 도서에 리뷰를 작성할 수 있다.
6	별점	리뷰에는 별점을 저장한다.
7	대여료	도서의 현재 대여료를 관리한다.
8	반납 상태	대여한 도서의 반납 여부를 관리한다.

활동 3. 독립적으로 관리할 대상 구분하기

항목	독립된 테이블	속성	이유
회원	✓		여러 회원을 독립적으로 관리해야 한다.
회원 이름		✓	회원을 설명하는 정보이다.
도서	✓		여러 도서의 정보를 관리해야 한다.
도서 제목		✓	도서를 설명하는 정보이다.
대여	✓		회원이 대여한 사건을 구분해 관리해야 한다.
대여일		✓	특정 대여에 속하는 정보이다.
대여도서	✓		한 번의 대여에 포함된 각각의 책을 관리한다.
반납 상태		✓	대여된 책의 상태를 나타낸다.
리뷰	✓		회원이 도서에 작성한 평가를 관리한다.

별점

✓ 리뷰를 설명하는 정보이다.

활동 4. 모든 정보를 하나의 표에 저장해 보기

대여번호	회원명	전화번호	도서명	저자	대여료
1001	홍길동	010-1111-1111	데이터베이스 입문	김정보	3000
1001	홍길동	010-1111-1111	웹 프로그래밍	이코딩	4000
1002	김민지	010-2222-2222	데이터베이스 입문	김정보	3000

반복되는 정보

- 홍길동의 회원명과 전화번호
- 데이터베이스 입문의 도서명과 저자
- 데이터베이스 입문의 대여료

활동 5. 하나의 표에서 발생하는 문제 찾기

문제 1

내 표에서 반복되는 정보:

회원명, 전화번호, 도서명, 저자, 대여료가 반복된다.

문제 2

여러 행을 수정해야 하는 정보:

회원 전화번호, 도서 제목, 저자, 현재 대여료

문제 3

일부 행만 수정할 때 발생하는 문제:

같은 회원의 전화번호가 행마다 다르게 저장되거나,
같은 도서의 대여료가 서로 다르게 표시될 수 있다.

문제 4

다른 데이터가 없어 저장하기 어려운 정보:

아직 한 번도 대여되지 않은 새 도서를 저장하기 어렵다.

문제 5

특정 행을 삭제했을 때 사라질 수 있는 정보:

도서의 마지막 대여 기록을 삭제하면서 도서 정보도 함께 사라질 수 있다.

활동 6. 데이터 중복으로 발생하는 문제 정리하기

1. 회원과 도서 정보가 여러 번 반복된다.
2. 전화번호나 도서 정보를 수정할 때 여러 행을 수정해야 한다.
3. 일부 행만 수정하면 서로 다른 값이 저장될 수 있다.
4. 대여 기록을 삭제하면서 도서나 회원 정보가 함께 사라질 수 있다.
5. 대여 기록이 없는 신규 도서를 저장하기 어렵다.

활동 7. 테이블을 나누는 기준 정하기

번호	테이블로 만들 대상	분리하는 이유
1	회원	회원 정보를 독립적으로 관리한다.
2	도서	도서 정보를 독립적으로 관리한다.
3	대여	회원이 대여한 사건을 기록한다.
4	대여도서	한 번의 대여에 포함된 각각의 도서를 기록한다.
5	리뷰	회원이 작성한 도서 평가를 기록한다.

2단계. 엔터티와 속성 설계하기

활동 8. 엔터티 정하기

주요 엔터티:

1. 회원
2. 도서
3. 대여
4. 대여도서
5. 리뷰

독립적으로 관리해야 하는 이유:

회원과 도서는 대여가 없어도 존재할 수 있다.
대여는 회원이 도서를 빌린 사건을 나타낸다.
대여도서는 한 대여에 포함된 각각의 도서를 관리한다.
리뷰는 회원이 도서에 작성한 평가 기록이다.

활동 9. 테이블 이름 정하기

한글 엔터티	영문 테이블 이름	관리할 내용
회원	<code>member</code>	회원 계정과 연락처
도서	<code>book</code>	도서 제목, 저자, 대여료와 재고
대여	<code>rental</code>	회원의 대여 일시와 상태
대여도서	<code>rental_item</code>	대여에 포함된 각각의 도서
리뷰	<code>review</code>	회원의 별점과 리뷰 내용

테이블 이름을 정할 때 주의한 점:

테이블이 저장하는 내용을 쉽게 알 수 있도록 단수형 영문 이름을 사용했다.
여러 단어를 사용할 때는 밑줄로 연결했다.
SQL 예약어와 혼동될 수 있는 이름은 피했다.

활동 10. 첫 번째 핵심 테이블 설계하기

테이블 이름:

`member`

저장할 내용	영문 열 이름	예상 데이터 형식	반드시 입력
회원번호	<code>member_id</code>	INT	예
로그인 아이디	<code>login_id</code>	VARCHAR	예
회원 이름	<code>member_name</code>	VARCHAR	예
전화번호	<code>phone</code>	VARCHAR	아니요
가입일	<code>joined_at</code>	DATETIME	예

활동 11. 첫 번째 테이블의 기본키 정하기

기본키 후보:

1. `member_id`
2. `login_id`
3. `phone`

최종 기본키:

`member_id`

선택한 이유:

회원번호는 회원마다 고유하게 부여할 수 있고,
회원이 정보를 수정해도 변경할 필요가 없기 때문이다.

중복 가능성:

`member_id`는 중복되지 않는다.

변경 가능성:

`member_id`는 일반적으로 변경하지 않는다.

로그인 아이디를 기본키로 사용하지 않은 이유:

로그인 아이디는 중복되면 안 되지만 운영 정책에 따라 변경될 수 있다.

따라서 기본키는 별도의 회원번호를 사용하고 로그인 아이디에는 **UNIQUE**를 적용할 수 있다.

활동 12. 두 번째 핵심 테이블 설계하기

테이블 이름:

book

저장할 내용	영문 열 이름	예상 데이터 형식	설명
도서번호	book_id	INT	도서를 구분하는 번호
도서 제목	title	VARCHAR	도서의 제목
저자	author	VARCHAR	책을 쓴 사람
현재 대여료	rental_fee	INT	현재 적용되는 대여 가격
보유수량	stock	INT	현재 보유한 책의 수
대여 가능 여부	is_available	CHAR	Y 또는 N으로 표시

기본키:

book_id

활동 13. 속성의 데이터 형식 정하기

속성	선택한 형식	이유
title	문자	한글과 영문 제목을 저장한다.
author	문자	저자의 이름을 저장한다.
rental_fee	숫자	금액 비교와 계산이 필요하다.

stock 숫자 재고 증감 계산이 필요하다.

is_available 상태값 Y와 N으로 대여 가능 여부를 표시한다.

계산이나 비교가 필요한 숫자 속성:

rental_fee, stock

날짜와 시간을 저장해야 하는 속성:

joined_at, rented_at, due_at, returned_at, created_at

활동 14. 세 번째 핵심 테이블 설계하기

테이블 이름:

rental

저장할 내용	영문 열 이름	예상 데이터 형식	역할
대여번호	rental_id	INT	대여 건을 구분
회원번호	member_id	INT	대여한 회원과 연결
대여일	rented_at	DATETIME	대여한 날짜와 시간
반납 예정일	due_at	DATETIME	반납해야 하는 날짜
대여상태	rental_status	VARCHAR	대여중, 반납완료 등

기본키:

rental_id

다른 테이블과 연결할 것으로 예상되는 열:

member_id

활동 15. 여러 대상을 연결해야 하는 상황 찾기

첫 번째 대상:

대여

여러 개 연결되는 두 번째 대상:

도서

구체적인 예:

대여번호 1001번에 데이터베이스 입문, 웹 프로그래밍,
파이썬 기초 세 권이 포함될 수 있다.

하나의 열에 여러 번호를 저장할 때 생기는 문제:

book_id에 "1, 2, 3"을 저장하면 개별 도서를 검색하거나
외래키로 연결하기 어렵다. 각 도서의 반납 상태나 대여료도 따로 저장하기 어렵다.

중간 테이블 필요 여부:

필요함

활동 16. 연결 또는 중간 테이블 설계하기

중간 테이블 이름:

rental_item

저장할 내용	영문 열 이름	예상 데이터 형식	역할
대여도서번호	rental_item_id	INT	각 대여도서 행을 구분
대여번호	rental_id	INT	어느 대여에 포함됐는지 표시
도서번호	book_id	INT	어떤 도서인지 표시

대여 당시 금액	<code>rental_price</code>	INT	대여 시점의 가격
반납일	<code>returned_at</code>	DATETIME	실제 반납한 날짜
반납상태	<code>return_status</code>	VARCHAR	대여중, 반납완료, 연체

기본키:

`rental_item_id`

첫 번째 테이블과 연결하는 열:

`rental_id`

두 번째 테이블과 연결하는 열:

`book_id`

활동 17. 시점에 따라 달라지는 데이터 찾기

시간이 지나면 변경될 수 있는 정보:

도서의 현재 대여료

과거의 값을 유지해야 하는 정보:

회원이 실제로 대여했을 당시의 대여 금액

현재 값과 과거 값을 분리해야 하는가?

예

구분	테이블	속성
현재 값	<code>book</code>	<code>rental_fee</code>

대여 당시 값 rental_i rental_pr
 tem ice

그 이유:

현재 대여료가 3,000원에서 4,000원으로 바뀌더라도
과거에 3,000원으로 대여한 기록은 3,000원으로 유지되어야 한다.

활동 18. 최종 속성 목록 완성하기

테이블 1: member

구분	열 이름
기본키	member_id
일반 속성	login_id
일반 속성	member_name
일반 속성	phone
일반 속성	joined_at

테이블 2: book

구분	열 이름
기본키	book_id
일반 속성	title
일반 속성	author
일반 속성	rental_fee

일반 속성	stock
일반 속성	is_available

테이블 3: rental

구분	열 이름
기본키	rental_id
외래키 후보	member_id
일반 속성	rented_at
일반 속성	due_at
일반 속성	rental_status

테이블 4: rental_item

구분	열 이름
기본키	rental_item_id
외래키 후보	rental_id
외래키 후보	book_id
일반 속성	rental_price
일반 속성	returned_at
일반 속성	return_status

추가 테이블: review

구분	열 이름
기본키	<code>review_id</code>
외래키 후보	<code>member_id</code>
외래키 후보	<code>book_id</code>
일반 속성	<code>rating</code>
일반 속성	<code>review_text</code>
일반 속성	<code>created_at</code>

3단계. 테이블 관계 설정하기

활동 19. 관계의 종류 구분하기

관계 종류	첫 번째 테이블	두 번째 테이블	관계 설명
1:1	member	member_profile	회원 한 명은 하나의 상세정보를 가진다.
1:N	member	rental	회원 한 명은 여러 번 대여할 수 있다.
N:M	rental	book	한 대여에 여러 도서가 포함되고 도서도 여러 대여에 포함된다.

`member_profile`을 만들지 않는다면 1:1 관계는 없다고 작성해도 된다.

활동 20. 첫 번째 관계 판단하기

연결할 두 테이블:

member — rental

첫 번째 테이블의 한 행은 두 번째 테이블의 몇 행과 연결되는가?

여러 행

두 번째 테이블의 한 행은 첫 번째 테이블의 몇 행과 연결되는가?

한 행

최종 관계:

1:N

관계 설명:

회원 한 명은 여러 번 도서를 대여할 수 있다.
대여 한 건은 반드시 한 명의 회원에게 속한다.

활동 21. 첫 번째 외래키 위치 정하기

1쪽 테이블:

member

N쪽 테이블:

rental

외래키를 저장할 테이블:

rental

외래키 이름:

member_id

참조하는 기본키:

member.member_id

연결:

rental.member_id
↓
member.member_id

활동 22. 두 번째 관계 판단하기

분석할 테이블:

rental — book

첫 번째 테이블의 한 행이 두 번째 테이블의 여러 행과 연결되는가?

예

두 번째 테이블의 한 행도 첫 번째 테이블의 여러 행과 연결되는가?

예

최종 관계:

N:M

이유:

한 번의 대여에 여러 권의 도서가 포함될 수 있고,
같은 도서도 여러 회원의 여러 대여 기록에 포함될 수 있기 때문이다.

활동 23. N:M 관계에서 저장할 추가 정보 찾기

N:M 관계:

rental N — M book

두 대상의 관계에서 발생하는 정보:

1. 대여 당시 금액
2. 실제 반납일
3. 반납 상태
4. 연체 여부

첫 번째 테이블에만 속하지 않는 이유:

하나의 대여에는 여러 도서가 있고 각각 반납일과 상태가 다를 수 있다.
따라서 대여 테이블에 하나의 값으로 저장할 수 없다.

두 번째 테이블에만 속하지 않는 이유:

같은 도서라도 대여 시기와 회원에 따라 대여 당시 금액과 반납 상태가 달라진다.
따라서 도서 자체의 고정 정보가 아니다.

활동 24. 중간 테이블로 N:M 관계 해결하기

중간 테이블 이름:

rental_item

첫 번째 관계:

rental 1 ——— N rental_item

두 번째 관계:

book 1 ——— N rental_item

중간 테이블 추가 전:

rental N ——— M book

중간 테이블 추가 후:

rental과 book의 N:M 관계를

rental 1:N rental_item과 book 1:N rental_item으로 나눈다.

활동 25. 중간 테이블의 역할 설명하기

중간 테이블 이름:

rental_item

저장하는 정보:

1. 어떤 대여에 포함되었는지
2. 어떤 도서가 대여되었는지
3. 대여 당시 금액
4. 실제 반납일
5. 반납 상태

한 문장 설명:

`rental_item`은 한 번의 대여에 포함된 각각의 도서를 기록하고,
대여와 도서의 **N:M** 관계를 두 개의 **1:N** 관계로 해결하는 테이블이다.

활동 26. 전체 테이블 관계 완성하기

member 1 — N rental

rental 1 — N rental_item

book 1 — N rental_item

member 1 — N review

book 1 — N review

활동 27. 기본키와 외래키 구분하기

테이블	열 이름	구분
member	<code>member_id</code>	기본키
book	<code>book_id</code>	기본키
rental	<code>rental_id</code>	기본키
rental	<code>member_id</code>	외래키
rental_item	<code>rental_id</code> <code>member_id</code>	기본키
rental_item	<code>rental_id</code>	외래키
rental_item	<code>book_id</code>	외래키

review	review_id	기본키
review	member_id	외래키
review	book_id	외래키

활동 28. 외래키 연결 완성하기

연결 1

rental.member_id
↓
member.member_id

연결 2

rental_item.rental_id
↓
rental.rental_id

연결 3

rental_item.book_id
↓
book.book_id

연결 4

review.member_id
↓
member.member_id

연결 5

review.book_id
↓
book.book_id

활동 29. 테이블 구조 카드 완성하기

member

member	
PK member_id	
login_id	
member_name	
phone	
joined_at	

book

book	
PK book_id	
title	
author	
rental_fee	
stock	
is_available	

rental

rental	
PK rental_id	
FK member_id	
rented_at	
due_at	
rental_status	

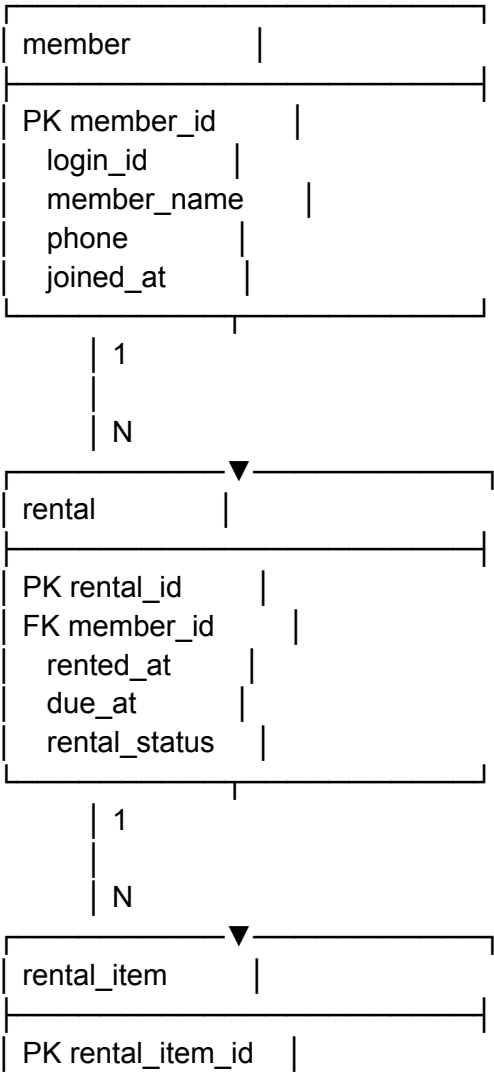
rental_item

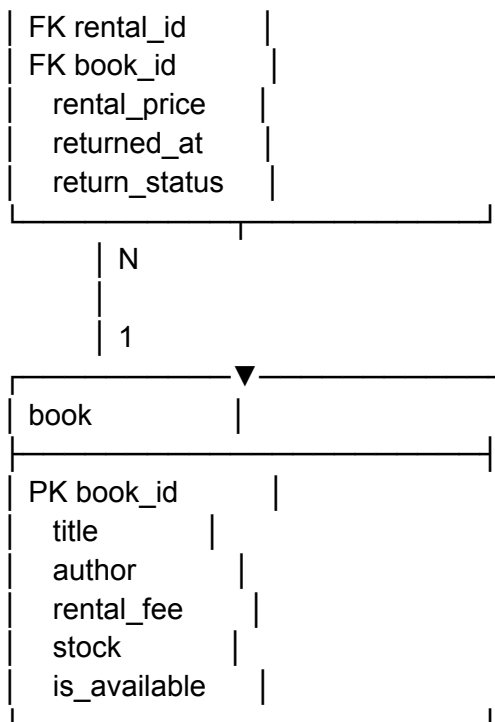
rental_item	
PK rental_item_id	
FK rental_id	
FK book_id	
rental_price	
returned_at	
return_status	

review

review	
PK review_id	
FK member_id	
FK book_id	
rating	
review_text	
created_at	

활동 30. 최종 ERD 작성하기





리뷰 관계:

member 1 — N review
book 1 — N review

모델 검증 활동

활동 31. 실제 사용 사례를 **ERD**에 적용하기

사용자:

홍길동

발생한 활동:

1. 홍길동이 데이터베이스 입문과 웹 프로그래밍을 한 번에 대여한다.
2. 홍길동이 다음 주에 파이썬 기초를 다시 대여한다.
3. 홍길동이 데이터베이스 입문에 별점 5점 리뷰를 작성한다.

예상 행 수:

테이블	저장되는 정보	행 수
member	홍길동 회원 정보	1
book	세 종류의 도서	3
rental	두 번의 대여	2
rental_item	첫 대여 2권, 두 번째 대여 1권	3
review	도서 리뷰	1

같은 정보가 불필요하게 반복되는가?

아니요

판단 이유:

회원과 도서 정보는 각각 한 번만 저장되고,
대여와 리뷰에서는 회원번호와 도서번호로 연결된다.

활동 32. 잘못된 설계 찾아 수정하기

설계 점검 1: 기본키

기본키로 사용한 값:

member_id

중복 가능성:

아니요

변경 가능성:

아니요

수정 필요 여부:

수정할 필요가 없다.

회원 이름을 기본키로 사용했다면:

동명이인이 있을 수 있으므로 `member_id`로 변경한다.

설계 점검 2: 반복 속성

여러 테이블에 반복하여 저장할 수 있는 정보:

회원 이름과 도서 제목

수정 방법:

`rental`이나 `review`에 회원 이름과 도서 제목을 직접 저장하지 않는다.
`member_id`와 `book_id` 외래키를 사용해 연결한다.

설계 점검 3: 한 열에 여러 값 저장

한 열에 여러 번호를 저장한 부분:

없음

잘못된 예:

```
rental.book_ids = "1, 2, 3"
```

수정 방법:

도서마다 `rental_item` 행을 따로 생성한다.

설계 점검 4: 누락된 관계 속성

관계에서 발생하는 정보:

1. 대여 당시 금액
2. 실제 반납일
3. 반납 상태

추가할 테이블 또는 속성:

`rental_item` 테이블에 `rental_price`, `returned_at`,
`return_status` 속성을 추가한다.

활동 33. 새로운 요구사항 반영하기

추가할 기능:

도서 카테고리 분류

운영 규칙:

하나의 카테고리에는 여러 권의 도서가 포함될 수 있다.

각 도서는 하나의 카테고리에 속한다.

필요한 새 테이블:

category

기본키:

category_id

주요 속성:

1. **category_name**
2. **description**
3. **is_active**

기존 테이블과의 관계:

1:N

외래키를 추가할 테이블:

book

외래키 이름:

category_id

수정된 관계:

category 1 — N book

최종 확인 문제

문제 1. 엔터티와 속성

엔터티:

도서

속성:

1. 도서 제목
2. 저자
3. 현재 대여료
4. 재고수량

설명:

도서는 독립적으로 관리할 대상이고,
제목과 저자 등은 도서를 설명하는 속성이다.

문제 2. 기본키

기본키:

`book_id`

필요한 이유:

도서 제목이 같을 수 있으므로 각 도서를 고유하게 구분하는 번호가 필요하다.

문제 3. 외래키

외래키:

`rental.member_id`

참조하는 기본키:

member.member_id

역할:

각 대여 기록을 실제 회원과 연결한다.

문제 4. 1:N 관계

member 1 — N rental

이유:

회원 한 명은 여러 번 도서를 대여할 수 있지만,
대여 한 건은 한 명의 회원에게만 속하기 때문이다.

문제 5. N:M 관계

rental N — M book

이유:

한 번의 대여에 여러 도서가 포함될 수 있고,
같은 도서도 여러 번의 대여에 포함될 수 있기 때문이다.

문제 6. 중간 테이블

중간 테이블:

rental_item

연결하는 두 테이블:

rental과 book

문제 7. 중간 테이블 속성

1. `rental_id`
 2. `book_id`
 3. `rental_price`
 4. `returned_at`
 5. `return_status`
-

문제 8. 현재 값과 당시 값

시간이 지나면 변경되는 현재 값:

`book.rental_fee`

과거 기록에 유지해야 하는 값:

`rental_item.rental_price`

분리해야 하는 이유:

현재 대여료가 변경되어도 과거 대여 기록에는 실제로 지불한 당시의 금액이 남아 있어야 하기 때문이다.

실습 마무리

나의 최종 데이터베이스 모델

항목	설계 결과
사이트 이름	복루프
전체 테이블 수	5개
기본키 수	5개
외래키 수	5개
1:1 관계 수	0개

1:N 관계 수	5개
직접 표현된 N:M 관계 수	0개
중간 테이블로 해결한 N:M 관계	rental과 book
중간 테이블 이름	rental_item

직접 표현된 N:M 관계가 0개인 이유:

rental과 book의 N:M 관계를 rental_item을 이용하여 두 개의 1:N 관계로 변환했기 때문이다.

나의 테이블 목록

번호	테이블 이름	관리하는 정보
1	member	회원 계정과 연락처
2	book	도서 정보와 현재 대여료
3	rental	회원의 대여 기록
4	rental_ item	대여에 포함된 각각의 도서
5	review	회원이 작성한 별점과 리뷰

오늘 알게 된 내용

1. 엔터티와 속성을 구분하여 테이블을 설계해야 한다.
2. 기본키는 각 행을 고유하게 구분한다.
3. 외래키는 서로 다른 테이블을 연결한다.
4. N:M 관계는 중간 테이블로 해결한다.
5. 현재 값과 과거 거래 값은 필요에 따라 분리해 저장한다.

설계하면서 변경한 내용

처음 생각한 구조:

회원, 도서, 대여 세 개 테이블만 만들려고 했다.
대여 테이블의 한 열에 여러 도서번호를 저장하려고 했다.

최종적으로 변경한 구조:

대여와 도서 사이에 **rental_item** 테이블을 추가했다.
리뷰는 별도의 **review** 테이블로 분리했다.

변경한 이유:

한 번의 대여에 여러 권의 도서가 포함될 수 있고,
각 도서의 대여료와 반납 상태를 따로 저장해야 하기 때문이다.

최종 자기 점검표 모범 확인

확인 항목	완료
구현하고 싶은 사이트를 정했다.	✓
사이트의 주요 기능과 사용자를 정했다.	✓
사이트 운영 규칙을 다섯 문장 이상 작성했다.	✓
요구사항에서 중요한 명사를 찾았다.	✓
엔터티와 속성을 구분했다.	✓
필요한 테이블을 결정했다.	✓
각 테이블의 영문 이름을 정했다.	✓
각 테이블의 주요 속성을 정했다.	✓
각 테이블의 기본키를 정했다.	✓
속성의 데이터 형식을 예상했다.	✓
1:1, 1:N, N:M 관계를 구분했다.	✓
필요한 중간 테이블을 설계했다.	✓
외래키의 위치를 정했다.	✓

- | | |
|---------------------|---|
| 기본키와 외래키의 연결을 표시했다. | ✓ |
| 실제 사용 사례를 모델에 적용했다. | ✓ |
| 잘못된 설계가 없는지 검토했다. | ✓ |
| 최종 ERD를 완성했다. | ✓ |